

Events that trigger workflows

You can configure your workflows to run when specific activity on GitHub happens, at a scheduled time, or when an event outside of GitHub occurs.

Copy markdown ▾

About events that trigger workflows [↗](#)

Workflow triggers are events that cause a workflow to run. For more information about how to use workflow triggers, see [Triggering a workflow](#).

Some events have multiple activity types. For these events, you can specify which activity types will trigger a workflow run. For more information about what each activity type means, see [Webhook events and payloads](#).

ⓘ

Note

Not all webhook events trigger workflows.

Like GitHub Actions workflows, agentic workflows can be triggered by repository events and schedules. For an example, see [Creating GitHub Agentic Workflows](#).

branch_protection_rule [↗](#)

branch_protection_rule	- created	Last commit on default branch	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

ⓘ

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when branch protection rules in the workflow repository are changed. For more information about branch protection rules, see [About protected branches](#). For information about the branch protection rule APIs, see [Branches](#) in the GraphQL API documentation or [REST API endpoints for branches and their settings](#).

For example, you can run a workflow when a branch protection rule has been `created` or `deleted` :

```
on:
  branch_protection_rule:
    types: [created, deleted]
```

check_run [↗](#)

`check_run`

- `created`
- `rerequested`
- `completed`

Last commit on default branch
Default branch

Webhook event payload

Activity types

`GITHUB_SHA`

`GITHUB_REF`

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- To prevent recursive workflows, this event does not trigger workflows if the check run's check suite was created by GitHub Actions or if the check suite's head SHA is associated with GitHub

Actions.

Runs your workflow when activity related to a check run occurs. A check run is an individual test that is part of a check suite. For information, see [Using the REST API to interact with checks](#). For information about the check run APIs, see [Checks](#) in the GraphQL API documentation or [REST API endpoints for check runs](#).

For example, you can run a workflow when a check run has been `rerequested` or `completed`.

```
on:
  check_run:
    types: [rerequested, completed]
```

check_suite

<code>check_suite</code>	<code>completed</code>	Last commit on default	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

 Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). Although only the `completed` activity type is supported, specifying the activity type will keep your workflow specific if more activity types are added in the future. By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- To prevent recursive workflows, this event does not trigger workflows if the check suite was created by GitHub Actions or if the check suite's head SHA is associated with GitHub Actions.

Runs your workflow when check suite activity occurs. A check suite is a collection of the check runs created for a specific commit. Check suites summarize the status and conclusion of the check runs that

are in the suite. For information, see [Using the REST API to interact with checks](#). For information about the check suite APIs, see [Checks](#) in the GraphQL API documentation or [REST API endpoints for check suites](#).

For example, you can run a workflow when a check suite has been `completed`.

```
on:
  check_suite:
    types: [completed]
```

create [↗](#)

<code>create</code>	Not applicable	Last commit on the created	Branch or tag created
Webhook event payload	Activity types	<code>GITHUB_SHA</code>	<code>GITHUB_REF</code>

ⓘ

Note

An event will not be created when you create more than three tags at once.

Runs your workflow when someone creates a Git reference (Git branch or tag) in the workflow's repository. For information about the APIs to create a Git reference, see [Git](#) in the GraphQL API documentation or [REST API endpoints for Git references](#).

For example, you can run a workflow when the `create` event occurs.

```
on:
  create
```

delete [↗](#)

<code>delete</code>	Not applicable	Last commit on default	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

 Note

- This event will only trigger a workflow run if the workflow file exists on the default branch.
- An event will not be created when you delete more than three tags at once.

Runs your workflow when someone deletes a Git reference (Git branch or tag) in the workflow's repository. For information about the APIs to delete a Git reference, see [Git](#) in the GraphQL API documentation or [REST API endpoints for Git references](#).

For example, you can run a workflow when the `delete` event occurs.

```
on:
  delete
```

deployment

<code>deployment</code>	Not applicable	Commit to be deployed	Branch or tag to be deployed (empty if created)
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

Runs your workflow when someone creates a deployment in the workflow's repository. Deployments created with a commit SHA may not have a Git ref. For information about the APIs to create a deployment, see [Deployments](#) in the GraphQL API documentation or [REST API endpoints for repositories](#).

For example, you can run a workflow when the `deployment` event occurs.

```
on:
  deployment
```

deployment_status [↗](#)

<code>deployment_status</code>	Not applicable	Commit to be deployed	Branch or tag to be deployed
Webhook event payload	Activity types	<code>GITHUB_SHA</code>	<code>GITHUB_REF</code>

Note

When a deployment status's state is set to `inactive`, a workflow run will not be triggered.

Runs your workflow when a third party provides a deployment status. Deployments created with a commit SHA may not have a Git ref. For information about the APIs to create a deployment status, see [Deployments](#) in the GraphQL API documentation or [REST API endpoints for deployments](#).

For example, you can run a workflow when the `deployment_status` event occurs.

```
on:
  deployment_status
```

discussion [↗](#)

`discussion`

- `created`
- `edited`
- `deleted`
- `transferred`
- `pinned`
- `unpinned`
- `labeled`
- `unlabeled`
- `locked`
- `unlocked`
- `category_changed`
- `answered`

Last commit on default branch
Default branch

Webhook event payload

Activity types

`GITHUB_SHA`

`GITHUB_REF`

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- Webhook events for GitHub Discussions are currently in public preview and subject to change.

Runs your workflow when a discussion in the workflow's repository is created or modified. For activity related to comments on a discussion, use the `discussion_comment` event. For more information about discussions, see [About discussions](#). For information about the GraphQL API, see [Discussions](#).

For example, you can run a workflow when a discussion has been `created`, `edited`, or `answered`.

```
on:
  discussion:
    types: [created, edited, answered]
```

`discussion_comment` [↗](#)

<code>discussion_comment</code>	- <code>created</code> <code>deleted</code>	Last commit on default branch	Default branch
Webhook event payload	Activity types	<code>GITHUB_SHA</code>	<code>GITHUB_REF</code>

 Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- Webhook events for GitHub Discussions are currently in public preview and subject to change.

Runs your workflow when a comment on a discussion in the workflow's repository is created or modified. For activity related to a discussion as opposed to comments on the discussion, use the `discussion` event. For more information about discussions, see [About discussions](#). For information about the GraphQL API, see [Discussions](#).

For example, you can run a workflow when a discussion comment has been `created` or `deleted`.

```
on:
  discussion_comment:
    types: [created, deleted]
```

fork

<code>pull_request</code>	Not applicable	Last commit on default branch	Default branch
Webhook event payload	Activity types	<code>GITHUB_SHA</code>	<code>GITHUB_REF</code>

 Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when someone forks a repository. For information about the REST API, see [REST API endpoints for forks](#).

For example, you can run a workflow when the `fork` event occurs.

```
on:
  fork
```

gollum [↗](#)

Not applicable	Not applicable	Last commit on default	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF



Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when someone creates or updates a Wiki page. For more information, see [About wikis](#).

For example, you can run a workflow when the `gollum` event occurs.

```
on:
  gollum
```

image_version [↗](#)

Not applicable	Not applicable	Last commit on default	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

Runs your workflow when a new version of a specified image becomes available for use. This event is typically triggered after a successful image version creation, allowing you to automate actions such as deployment or notifications in response to new image versions.

This event supports glob patterns for both image names and versions. The example below triggers when a new image version matches any of the specified name and version combinations. For example,

`[ "MyNewImage", 1.0.0 ]`, `[ "MyNewImage", 2.53.0 ]`, `[ "MyOtherImage", 1.0.0 ]`, and `[ "MyOtherImage", 2.0.0 ]`.

```
on:
  image_version:
    names:
      - "MyNewImage"
      - "MyOtherImage"
    versions:
      - 1.*
      - 2.*
```

issue_comment

`issue_comment`

- `created`

Last commit on default
branch

Default branch

Webhook event payload

Activity types

`GITHUB_SHA`

`GITHUB_REF`

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when an issue or pull request comment is created, edited, or deleted. For information about the issue comment APIs, see [Issues](#) in the GraphQL API documentation or [Webhook events and payloads](#) in the REST API documentation.

For example, you can run a workflow when an issue or pull request comment has been `created` or `deleted`.

```
on:
  issue_comment:
    types: [created, deleted]
```

`issue_comment` on issues only or pull requests only [↗](#)

The `issue_comment` event occurs for comments on both issues and pull requests. You can use the `github.event.issue.pull_request` property in a conditional to take different action depending on whether the triggering object was an issue or pull request.

For example, this workflow will run the `pr_commented` job only if the `issue_comment` event originated from a pull request. It will run the `issue_commented` job only if the `issue_comment` event originated from an issue.

```
on: issue_comment

jobs:
  pr_commented:
    # This job only runs for pull request comments
    name: PR comment
    if: ${[ github.event.issue.pull_request ]}
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo A comment on PR $NUMBER
        env:
          NUMBER: ${[ github.event.issue.number ]}

  issue_commented:
    # This job only runs for issue comments
    name: Issue comment
    if: ${[ !github.event.issue.pull_request ]}
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo A comment on issue $NUMBER
        env:
          NUMBER: ${[ github.event.issue.number ]}
```

issues

issues	- opened - edited - deleted - transferred - pinned - unpinned - closed - reopened - assigned - unassigned - labeled - unlabeled - locked - unlocked - milestoned - demilestoned - typed - untyped - filtered	Last commit on default branch	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when an issue in the workflow's repository is created or modified. For activity related to comments in an issue, use the `issue_comment` event. For more information about issues, see [About issues](#). For information about the issue APIs, see [Issues](#) in the GraphQL API documentation or [REST API endpoints for issues](#).

For example, you can run a workflow when an issue has been `opened` , `edited` , or `milestoned` .

🏠

Home / GitHub Actions / Reference / Workflows and actions / Events that trigger workflows

GitHub Actions

Get started

Concepts

How-tos

Reference

Workflows and actions

Workflow syntax

Events that trigger workflows

Workflow commands

Variables

Expressions

Contexts

Deployments and environments

Dependency caching

Reusing workflow configurations

Metadata syntax

Workflow cancellation

Dockerfile support

Runners

Security

Limits

GitHub Actions Importer

Tutorials

types: [opened, edited, milestoned]

You can also run a workflow when an issue field value is set, changed, or cleared. The `field_added` activity type fires both when a field value is initially set and when an existing value is updated. The `field_removed` activity type fires when a field value is cleared.

```
on:
  issues:
    types: [field_added, field_removed]
```

label

label	- created edited	Last commit on default branch	Default branch
Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when a label in your workflow's repository is created or modified. For more information about labels, see [Managing labels](#). For information about the label APIs, see [Issues](#) in the GraphQL API documentation or [REST API endpoints for labels](#).

If you want to run your workflow when a label is added to or removed from an issue, pull request, or discussion, use the `labeled` or `unlabeled` activity types for the `issues`, `pull_request`, `pull_request_target`, or `discussion` events instead.

deployment_status

discussion

discussion_comment

fork

gollum

image_version

issue_comment

issues

label

merge_group

milestone

page_build

public

pull_request

pull_request_comment (use is...

pull_request_review

pull_request_review_comment

pull_request_target

push

registry_package

release

repository_dispatch

https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#pull_request

13/48

For example, you can run a workflow when a label has been `created` or `deleted`.

```
on:
  label:
    types: [created, deleted]
```

merge_group [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>merge_group</code>	<code>checks_requested</code>	SHA of the merge group	Ref of the merge group

Note

- More than one activity type triggers this event. Although only the `checks_requested` activity type is supported, specifying the activity type will keep your workflow specific if more activity types are added in the future. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- If your repository uses GitHub Actions to perform required checks on pull requests in your repository, you need to update the workflows to include the `merge_group` event as an additional trigger. Otherwise, status checks will not be triggered when you add a pull request to a merge queue. The merge will fail as the required status check will not be reported. The `merge_group` event is separate from the `pull_request` and `push` events.

Runs your workflow when a pull request is added to a merge queue, which adds the pull request to a merge group. For more information see [Merging a pull request with a merge queue](#).

You can use the `branches` or `branches-ignore` filter to configure your workflow to run only for merge groups that target specific branches. For more information, see [Workflow syntax for GitHub Actions](#).

For example, the following workflow runs when the `checks_requested` activity occurs for a merge group that targets `main`.

```
on:
  pull_request:
    branches: [main]
  merge_group:
    types: [checks_requested]
    branches: [main]
```

milestone

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>milestone</code>	- <code>created</code> - <code>closed</code> - <code>opened</code> - <code>edited</code> - <code>deleted</code>	Last commit on default branch	Default branch

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when a milestone in the workflow's repository is created or modified. For more information about milestones, see [About milestones](#). For information about the milestone APIs, see [Issues](#) in the GraphQL API documentation or [REST API endpoints for milestones](#).

If you want to run your workflow when an issue is added to or removed from a milestone, use the `milestoned` or `demilestoned` activity types for the [issues](#) event instead.

For example, you can run a workflow when a milestone has been `opened` or `deleted`.

```
on:
  milestone:
    types: [opened, deleted]
```

page_build [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
page_build	Not applicable	Last commit on default branch	Default branch

Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when someone pushes to a branch that is the publishing source for GitHub Pages, if GitHub Pages is enabled for the repository. For more information about GitHub Pages publishing sources, see [Configuring a publishing source for your GitHub Pages site](#). For information about the REST API, see [REST API endpoints for repositories](#).

For example, you can run a workflow when the `page_build` event occurs.

```
on:
  page_build
```

public [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
public	Not applicable	Last commit on default branch	Default branch

Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when your workflow's repository changes from private to public. For information about the REST API, see [REST API endpoints for repositories](#).

For example, you can run a workflow when the `public` event occurs.

```
on:
  public
```

pull_request

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
pull_request	- assigned - unassigned - labeled - unlabeled - opened - edited - closed - reopened - synchronize - converted_to_draft - locked - unlocked - enqueued - dequeued - milestoned - demilestoned - ready_for_review - review_requested - review_request_removed - auto_merge_enabled - auto_merge_disabled	Last merge commit on the GITHUB_REF branch	PR merge branch refs/pull/PULL_REQUEST_NUMBER/merge

 Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, a workflow only runs when a pull_request event's activity type is opened , synchronize , or reopened . To trigger workflows by different activity types, use the types keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- Workflows will not run on pull_request activity if the pull request has a merge conflict. The merge conflict must be resolved first. Conversely, workflows with the pull_request_target

event will run even if the pull request has a merge conflict. Before using the

`pull_request_target` trigger, you should be aware of the security risks. For more information, see `pull_request_target`.

- The `pull_request` webhook event payload is empty for merged pull requests and pull requests that come from forked repositories.
- When a pull request is created or updated by a workflow using `GITHUB_TOKEN`, `pull_request` events with the `opened`, `synchronize`, or `reopened` activity types create workflow runs that require approval. A user with write access to the repository can approve these runs from the pull request page. With the exception of `workflow_dispatch` and `repository_dispatch`, other `GITHUB_TOKEN`-triggered events do not create workflow runs at all.
- The value of `GITHUB_REF` varies for a closed pull request depending on whether the pull request has been merged or not. If a pull request was closed but not merged, it will be `refs/pull/PULL_REQUEST_NUMBER/merge`. If a pull request was closed as a result of being merged, it will be the fully qualified `ref` of the branch it was merged into, for example `/refs/heads/main`.

Runs your workflow when activity on a pull request in the workflow's repository occurs. For example, if no activity types are specified, the workflow runs when a pull request is opened or reopened or when the head branch of the pull request is updated. For activity related to pull request reviews, pull request review comments, or pull request comments, use the `pull_request_review`, `pull_request_review_comment`, or `issue_comment` events instead. For information about the pull request APIs, see [Pull requests](#) in the GraphQL API documentation or [REST API endpoints for pull requests](#).

Note that `GITHUB_SHA` for this event is the last merge commit of the pull request merge branch. If you want to get the commit ID for the last commit to the head branch of the pull request, use `github.event.pull_request.head.sha` instead. For more information about merge branches, see [Pull requests](#).

How the merge branch affects your workflow [↗](#)

For open, mergeable pull requests, workflows triggered by the `pull_request` event set `GITHUB_REF` to the merge branch. Because `actions/checkout` uses `GITHUB_REF` by default, it checks out the merge branch. Your CI tests run against the merged result, not just the head branch alone:

- `GITHUB_REF` is set to `refs/pull/PULL_REQUEST_NUMBER/merge`
- `GITHUB_SHA` is the SHA of the merge commit on the merge branch

To test only the head branch commits without simulating a merge, check out the head branch using `github.event.pull_request.head.sha` in your workflow.

For example, you can run a workflow when a pull request has been opened or reopened.

```
on:
  pull_request:
    types: [opened, reopened]
```

You can use the event context to further control when jobs in your workflow will run. For example, this workflow will run when a review is requested on a pull request, but the `specific_review_requested` job will only run when a review by `octo-team` is requested.

```
on:
  pull_request:
    types: [review_requested]
jobs:
  specific_review_requested:
    runs-on: ubuntu-latest
    if: ${[ github.event.requested_team.name == 'octo-team' ]}
    steps:
      - run: echo 'A review from octo-team was requested'
```

Running your `pull_request` workflow based on the head or base branch of a pull request [↗](#)

You can use the `branches` or `branches-ignore` filter to configure your workflow to only run on pull requests that target specific branches. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when someone opens a pull request that targets a branch whose name starts with `releases/`:

```
on:
  pull_request:
    types:
      - opened
    branches:
      - 'releases/**'
```

Note

If you use both the `branches` filter and the `paths` filter, the workflow will only run when both filters are satisfied. For example, the following workflow will only run when a pull request that includes a change to a JavaScript (`.js`) file is opened on a branch whose name starts with `releases/`:

```
on:
  pull_request:
    types:
      - opened
    branches:
      - 'releases/**'
    paths:
      - '**.js'
```

To run a job based on the pull request's head branch name (as opposed to the pull request's base branch name), use the `github.head_ref` context in a conditional. For example, this workflow will run whenever a pull request is opened, but the `run_if` job will only execute if the head of the pull request is a branch whose name starts with `releases/`:

```
on:
  pull_request:
    types:
      - opened
jobs:
  run_if:
    if: startsWith(github.head_ref, 'releases/')
    runs-on: ubuntu-latest
    steps:
      - run: echo "The head of this PR starts with 'releases/'"
```

Running your `pull_request` workflow based on files changed in a pull request [↗](#)

You can also configure your workflow to run when a pull request changes specific files. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when a pull request includes a change to a JavaScript file (`.js`):

```
on:
  pull_request:
    paths:
      - '**.js'
```

Note

If you use both the `branches` filter and the `paths` filter, the workflow will only run when both filters are satisfied. For example, the following workflow will only run when a pull request that includes a change to a JavaScript (`.js`) file is opened on a branch whose name starts with `releases/` :

```
on:
  pull_request:
    types:
      - opened
    branches:
      - 'releases/**'
    paths:
      - '**.js'
```

Running your `pull_request` workflow when a pull request merges [↗](#)

When a pull request merges, the pull request is automatically closed. To run a workflow when a pull request merges, use the `pull_request` `closed` event type along with a conditional that checks the `merged` value of the event. For example, the following workflow will run whenever a pull request closes. The `if_merged` job will only run if the pull request was also merged.

```
on:
  pull_request:
    types:
      - closed

jobs:
  if_merged:
    if: github.event.pull_request.merged == true
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo The PR was merged
```

Workflows in forked repositories [↗](#)

Workflows don't run in forked repositories by default. You must enable GitHub Actions in the **Actions** tab of the forked repository.

With the exception of `GITHUB_TOKEN` , secrets are not passed to the runner when a workflow is triggered from a forked repository. The `GITHUB_TOKEN` has read-only permissions in pull requests from forked repositories. For more information, see [Use GITHUB_TOKEN for authentication in workflows](#).

Pull request events for forked repositories [↗](#)

For pull requests from a forked repository to the base repository, GitHub sends the `pull_request` , `issue_comment` , `pull_request_review_comment` , `pull_request_review` , and `pull_request_target` events to the base repository. No pull request events occur on the forked repository.

When a first-time contributor submits a pull request to a public repository, a maintainer with write access may need to approve running workflows on the pull request. For more information, see [Approving workflow runs from forks](#).

For pull requests from a forked repository to a private repository, workflows only run when they are enabled, see [Managing GitHub Actions settings for a repository](#).

[i](#) Note

Workflows triggered by Dependabot pull requests are treated as though they are from a forked repository, and are also subject to these restrictions.

`pull_request_comment` (use `issue_comment`) [↗](#)

To run your workflow when a comment on a pull request (not on a pull request's diff) is created, edited, or deleted, use the `issue_comment` event. For activity related to pull request reviews or pull request review comments, use the `pull_request_review` or `pull_request_review_comment` events.

`pull_request_review` [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>pull_request_review</code>	- submitted - edited - dismissed	Last merge commit on the <code>GITHUB_REF</code> branch	PR merge branch <code>refs/pull/<code>PULL_REQUEST_NUMBER</code>/merge</code>

Note

More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).

Runs your workflow when a pull request review is submitted, edited, or dismissed. A pull request review is a group of pull request review comments in addition to a body comment and a state. For activity related to pull request review comments or pull request comments, use the `pull_request_review_comment` or `issue_comment` events instead. For information about the pull request review APIs, see [Pull requests](#) in the GraphQL API documentation or [REST API endpoints for pull requests](#).

For example, you can run a workflow when a pull request review has been `edited` or `dismissed`.

```
on:
  pull_request_review:
    types: [edited, dismissed]
```

Running a workflow when a pull request is approved

To run your workflow when a pull request has been approved, you can trigger your workflow with the `submitted` type of `pull_request_review` event, then check the review state with the `github.event.review.state` property. For example, this workflow will run whenever a pull request review is submitted, but the `approved` job will only run if the submitted review is an approving review:

```
on:
  pull_request_review:
    types: [submitted]

jobs:
  approved:
    if: github.event.review.state == 'approved'
    runs-on: ubuntu-latest
    steps:
      - run: echo "This PR was approved"
```

Workflows in forked repositories

Workflows don't run in forked repositories by default. You must enable GitHub Actions in the **Actions** tab of the forked repository.

With the exception of `GITHUB_TOKEN`, secrets are not passed to the runner when a workflow is triggered from a forked repository. The `GITHUB_TOKEN` has read-only permissions in pull requests from forked repositories. For more information, see [Use GITHUB_TOKEN for authentication in workflows](#).

Pull request events for forked repositories [↗](#)

For pull requests from a forked repository to the base repository, GitHub sends the `pull_request`, `issue_comment`, `pull_request_review_comment`, `pull_request_review`, and `pull_request_target` events to the base repository. No pull request events occur on the forked repository.

When a first-time contributor submits a pull request to a public repository, a maintainer with write access may need to approve running workflows on the pull request. For more information, see [Approving workflow runs from forks](#).

For pull requests from a forked repository to a private repository, workflows only run when they are enabled, see [Managing GitHub Actions settings for a repository](#).

Note

Workflows triggered by Dependabot pull requests are treated as though they are from a forked repository, and are also subject to these restrictions.

`pull_request_review_comment` [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>pull_request_review_comment</code>	<code>created</code> <code>edited</code> <code>deleted</code>	Last merge commit on the <code>GITHUB_REF</code> branch	PR merge branch <code>refs/pull/<code>PULL_REQUEST_NUMBER</code>/merge</code>

Note

More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event.

You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).

Runs your workflow when a pull request review comment is modified. A pull request review comment is a comment on a pull request's diff. For activity related to pull request reviews or pull request comments, use the `pull_request_review` or `issue_comment` events instead. For information about the pull request review comment APIs, see [Pull requests](#) in the GraphQL API documentation or [REST API endpoints for pull requests](#).

For example, you can run a workflow when a pull request review comment has been `created` or `deleted`.

```
on:
  pull_request_review_comment:
    types: [created, deleted]
```

Workflows in forked repositories [↗](#)

Workflows don't run in forked repositories by default. You must enable GitHub Actions in the **Actions** tab of the forked repository.

With the exception of `GITHUB_TOKEN`, secrets are not passed to the runner when a workflow is triggered from a forked repository. The `GITHUB_TOKEN` has read-only permissions in pull requests from forked repositories. For more information, see [Use GITHUB_TOKEN for authentication in workflows](#).

Pull request events for forked repositories [↗](#)

For pull requests from a forked repository to the base repository, GitHub sends the `pull_request`, `issue_comment`, `pull_request_review_comment`, `pull_request_review`, and `pull_request_target` events to the base repository. No pull request events occur on the forked repository.

When a first-time contributor submits a pull request to a public repository, a maintainer with write access may need to approve running workflows on the pull request. For more information, see [Approving workflow runs from forks](#).

For pull requests from a forked repository to a private repository, workflows only run when they are enabled, see [Managing GitHub Actions settings for a repository](#).

[i](#) Note

Workflows triggered by Dependabot pull requests are treated as though they are from a forked repository, and are also subject to these restrictions.

pull_request_target [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
pull_request	- assigned - unassigned - labeled - unlabeled - opened - edited - closed - reopened - synchronize - converted_to_draft - locked - unlocked - enqueued - dequeued - milestoned - demilestoned - ready_for_review - review_requested - review_request_removed - auto_merge_enabled - auto_merge_disabled	Last commit on default branch	Default branch

Note

More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, a workflow only runs when a pull_request_target event's activity type is opened , synchronize , or reopened . To trigger workflows by different activity types, use the types keyword. For more information, see [Workflow syntax for GitHub Actions](#).

Runs your workflow when activity on a pull request in the workflow's repository occurs. For example, if no activity types are specified, the workflow runs when a pull request is opened or reopened or when the head

branch of the pull request is updated.

This event runs in the context of the default branch of the base repository, rather than in the context of the merge commit, as the `pull_request` event does. This prevents execution of unsafe code from the head of the pull request that could alter your repository or steal any secrets you use in your workflow. This event allows your workflow to do things like label or comment on pull requests from forks. Avoid using this event if you need to build or run code from the pull request.

To ensure repository security, branches with names that match certain patterns (such as those which look similar to SHAs) may not trigger workflows with the `pull_request_target` event.

⚠ Warning

Running untrusted code on the `pull_request_target` trigger may lead to security vulnerabilities. These vulnerabilities include cache poisoning and granting unintended access to write privileges or secrets. To learn how to use this trigger safely, see [Securely using `pull\_request\_target`](#). For more details on the underlying risks, see [Secure use reference](#) and [Preventing pwn requests](#) from GitHub Security Lab.

For example, you can run a workflow when a pull request has been `assigned`, `opened`, `synchronize`, or `reopened`.

```
on:
  pull_request_target:
    types: [assigned, opened, synchronize, reopened]
```

Running your `pull_request_target` workflow based on the head or base branch of a pull request [↗](#)

You can use the `branches` or `branches-ignore` filter to configure your workflow to only run on pull requests that target specific branches. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when someone opens a pull request that targets a branch whose name starts with `releases/`:

```
on:
  pull_request_target:
    types:
      - opened
```

```
branches:
  - 'releases/**'
```

Note

If you use both the `branches` filter and the `paths` filter, the workflow will only run when both filters are satisfied. For example, the following workflow will only run when a pull request that includes a change to a JavaScript (`.js`) file is opened on a branch whose name starts with `releases/` :

```
on:
  pull_request_target:
    types:
      - opened
    branches:
      - 'releases/**'
    paths:
      - '**.js'
```

To run a job based on the pull request's head branch name (as opposed to the pull request's base branch name), use the `github.head_ref` context in a conditional. For example, this workflow will run whenever a pull request is opened, but the `run_if` job will only execute if the head of the pull request is a branch whose name starts with `releases/` :

```
on:
  pull_request_target:
    types:
      - opened
jobs:
  run_if:
    if: startsWith(github.head_ref, 'releases/')
    runs-on: ubuntu-latest
    steps:
      - run: echo "The head of this PR starts with 'releases/'"
```

Running your `pull_request_target` workflow based on files changed in a pull request

You can use the `paths` or `paths-ignore` filter to configure your workflow to run when a pull request changes specific files. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when a pull request includes a change to a JavaScript file (`.js`):

```
on:
  pull_request_target:
    paths:
      - '**.js'
```

Note

If you use both the `branches` filter and the `paths` filter, the workflow will only run when both filters are satisfied. For example, the following workflow will only run when a pull request that includes a change to a JavaScript (`.js`) file is opened on a branch whose name starts with `releases/` :

```
on:
  pull_request_target:
    types:
      - opened
    branches:
      - 'releases/**'
    paths:
      - '**.js'
```

Running your `pull_request_target` workflow when a pull request merges [↗](#)

When a pull request merges, the pull request is automatically closed. To run a workflow when a pull request merges, use the `pull_request_target` `closed` event type along with a conditional that checks the `merged` value of the event. For example, the following workflow will run whenever a pull request closes. The `if_merged` job will only run if the pull request was also merged.

```
on:
  pull_request_target:
    types:
      - closed

jobs:
  if_merged:
    if: github.event.pull_request.merged == true
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo The PR was merged
```

push [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>push</code>	Not applicable	Tip commit pushed to the ref. When you delete a branch, the SHA in the workflow run (and its associated refs) reverts to the default branch of the repository.	Updated ref

Note

- The webhook payload available to GitHub Actions does not include the `added`, `removed`, and `modified` attributes in the `commit` object. You can retrieve the full commit object using the API. For information, see [Commits](#) in the GraphQL API documentation or [REST API endpoints for commits](#).
- Events will not be created if more than 5,000 branches are pushed at once. Events will not be created for tags when more than three tags are pushed at once.

Runs your workflow when you push a commit or tag, or when you create a repository from a template. This includes workflows that are not merged into the default branch. For more information, see [Events that trigger workflows](#).

For example, you can run a workflow when the `push` event occurs.

```
on:
  push
```

Note

When a `push` webhook event triggers a workflow run, the Actions UI's "pushed by" field shows the account of the pusher and not the author or committer. However, if the changes are pushed to a repository using SSH authentication with a deploy key, then the "pushed by" field will be the repository admin who verified the deploy key when it was added it to a repository.

Running your workflow only when a push to specific branches occurs [↗](#)

You can use the `branches` or `branches-ignore` filter to configure your workflow to only run when specific branches are pushed. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when someone pushes to `main` or to a branch that starts with `releases/`.

```
on:
  push:
    branches:
      - 'main'
      - 'releases/**'
```

Note

If you use both the `branches` filter and the `paths` filter, the workflow will only run when both filters are satisfied. For example, the following workflow will only run when a push that includes a change to a JavaScript (`.js`) file is made to a branch whose name starts with `releases/`:

```
on:
  push:
    branches:
      - 'releases/**'
    paths:
      - '**.js'
```

Running your workflow only when a push of specific tags occurs [↗](#)

You can use the `tags` or `tags-ignore` filter to configure your workflow to only run when specific tags are pushed. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when someone pushes a tag that starts with `v1.`.

```
on:
  push:
    tags:
      - v1.**
```

Running your workflow only when a push affects specific files [↗](#)

You can use the `paths` or `paths-ignore` filter to configure your workflow to run when a push to specific files occurs. For more information, see [Workflow syntax for GitHub Actions](#).

For example, this workflow will run when someone pushes a change to a JavaScript file (`.js`):

```
on:
  push:
    paths:
      - '**.js'
```

registry_package [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>registry_package</code>	- <code>published</code> - <code>updated</code>	Commit of the published package	Branch or tag of the published package

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- When pushing multi-architecture container images, this event occurs once per manifest, so you might observe your workflow triggering multiple times. To mitigate this, and only run your workflow job for the event that contains the actual image tag information, use a conditional:

```
jobs:
  job_name:
    if: $true
```

Runs your workflow when activity related to GitHub Packages occurs in your repository. For more information, see [GitHub Packages Documentation](#).

For example, you can run a workflow when a new package version has been `published`.

```
on:
  registry_package:
    types: [published]
```

release [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>release</code>	<code>published</code> <code>unpublished</code> <code>created</code> <code>edited</code> <code>deleted</code> <code>prereleased</code> <code>released</code>	Last commit in the tagged release	Tag ref of release <code>refs/tags/<tag_name></code>

Note

- More than one activity type triggers this event. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- Workflows are not triggered for the `created`, `edited`, or `deleted` activity types for draft releases. When you create your release through the GitHub UI, your release may automatically be saved as a draft.
- The `prereleased` type will not trigger for pre-releases published from draft releases, but the `published` type will trigger. If you want a workflow to run when stable *and* pre-releases publish, subscribe to `published` instead of `released` and `prereleased`.

Runs your workflow when release activity in your repository occurs. For information about the release APIs, see [Releases](#) in the GraphQL API documentation or [REST API endpoints for releases and release](#)

[assets](#) in the REST API documentation.

For example, you can run a workflow when a release has been `published`.

```
on:
  release:
    types: [published]
```

repository_dispatch [↗](#)

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
repository_dispatch	Custom	Last commit on default branch	Default branch

ⓘ

Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

You can use the GitHub API to trigger a webhook event called `repository_dispatch` when you want to trigger a workflow for activity that happens outside of GitHub. For more information, see [REST API endpoints for repositories](#).

When you make a request to create a `repository_dispatch` event, you must specify an `event_type` to describe the activity type. By default, all `repository_dispatch` activity types trigger a workflow to run. You can use the `types` keyword to limit your workflow to run when a specific `event_type` value is sent in the `repository_dispatch` webhook payload.

```
on:
  repository_dispatch:
    types: [test_result]
```

ⓘ

Note

The `event_type` value is limited to 100 characters.

Any data that you send through the `client_payload` parameter will be available in the `github.event` context in your workflow. For example, if you send this request body when you create a repository dispatch event:

```
{
  "event_type": "test_result",
  "client_payload": {
    "passed": false,
    "message": "Error: timeout"
  }
}
```

then you can access the payload in a workflow like this:

```
on:
  repository_dispatch:
    types: [test_result]

jobs:
  run_if_failure:
    if: ${{ !github.event.client_payload.passed }}
    runs-on: ubuntu-latest
    steps:
      - env:
          MESSAGE: ${{ github.event.client_payload.message }}
        run: echo $MESSAGE
```

 Note

- The maximum number of top-level properties in `client_payload` is 10.
- The payload can contain a maximum of 65,535 characters.

schedule

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
Not applicable	Not applicable	Last commit on default branch	Default branch

Note

- The `schedule` event can be delayed during periods of high loads of GitHub Actions workflow runs. High load times include the start of every hour. If the load is sufficiently high enough, some queued jobs may be dropped. To decrease the chance of delay, schedule your workflow to run at a different time of the hour.
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- Scheduled workflows will only run on the default branch.
- In a public repository, scheduled workflows are automatically disabled when no repository activity has occurred in 60 days. For information on re-enabling a disabled workflow, see [Disabling and enabling a workflow](#).

The `schedule` event allows you to trigger a workflow at a scheduled time.

Example:

```
on:
  schedule:
    - cron: "15 4,5 * * *"
```

Use [POSIX cron syntax](#) to schedule workflows to run at specific times. By default, scheduled workflows run in UTC. You can optionally specify a timezone using an [IANA timezone string](#) for timezone-aware scheduling. Scheduled workflows run on the latest commit on the default branch. The shortest interval you can run scheduled workflows is once every 5 minutes.

Note

For schedules that set `timezone` to a time zone that observes daylight saving time (DST), during DST spring-forward transitions, scheduled workflows in skipped hours advance to the next valid time. For example, a 2:30 AM schedule advances to 3:00 AM.

Cron syntax has five fields separated by a space, and each field represents a unit of time.

```
minute (0 - 59)
hour (0 - 23)
day of the month (1 - 31)
month (1 - 12 or JAN-DEC)
day of the week (0 - 6 or SUN-SAT)
```

```
| | | | |
* * * * *
```

You can use these operators in any of the five fields:

Operator	Description	Example
*	Any value	<code>15 * * * *</code> runs at every minute 15 of every hour of every day.
,	Value list separator	<code>2,10 4,5 * * *</code> runs at minute 2 and 10 of the 4th and 5th hour of every day.
-	Range of values	<code>30 4-6 * * *</code> runs at minute 30 of the 4th, 5th, and 6th hour.
/	Step values	<code>20/15 * * * *</code> runs every 15 minutes starting from minute 20 through 59 (minutes 20, 35, and 50).

This example triggers the workflow to run at 5:30 AM in the America/New_York timezone every Monday through Friday:

```
on:
  schedule:
    - cron: '30 5 * * 1-5'
      timezone: "America/New_York"
```

A single workflow can be triggered by multiple `schedule` events. Access the `schedule` event that triggered the workflow through the `github.event.schedule` context. This example triggers the workflow to run at 5:30 UTC every Monday-Thursday, and 17:30 UTC on Tuesdays and Thursdays, but skips the `Not on Monday or Wednesday` step on Monday and Wednesday.

```
on:
  schedule:
    - cron: '30 5 * * 1,3'
    - cron: '30 5,17 * * 2,4'

jobs:
  test_schedule:
```

```
runs-on: ubuntu-latest
steps:
  - name: Not on Monday or Wednesday
    if: github.event.schedule != '30 5 * * 1,3'
    run: echo "This step will be skipped on Monday and Wednesday"
  - name: Every time
    run: echo "This step will always run"
```

Note

GitHub Actions does not support the non-standard syntax `@yearly`, `@monthly`, `@weekly`, `@daily`, `@hourly`, and `@reboot`.

You can use [crontab guru](#) to help generate your cron syntax and confirm what time it will run. To help you get started, there is also a list of [crontab guru examples](#).

actor for scheduled workflows

Certain repository events change the `actor` associated with the workflow. For example, a user who changes the default branch of the repository, which changes the branch on which scheduled workflows run, becomes `actor` for those scheduled workflows.

For a deactivated scheduled workflow, if a user with `write` permissions to the repository makes a commit that changes the `cron` schedule on the workflow, the workflow will be reactivated, and that user will become the `actor` associated with any workflow runs.

Notifications for scheduled workflows are sent to the user who last modified the cron syntax in the workflow file. For more information, see [Notifications for workflow runs](#).

Note

For an enterprise with Enterprise Managed Users, triggering a scheduled workflow requires that the status of the `actor` user account associated with the workflow is currently active (i.e. not suspended or deleted).

- Scheduled workflows will not run if the last `actor` associated with the scheduled workflow has been deprovisioned by the Enterprise Managed User identity provider (IdP). However, if the last `actor` Enterprise Managed User has not been deprovisioned by the IdP, and has only been removed as a member from a given organization in the enterprise, scheduled workflows will still run with that user set as the `actor`.

- Similarly, for an enterprise without Enterprise Managed Users, removing a user from an organization will not prevent scheduled workflows which had that user as their `actor` from running.
- Thus, the *user account's* status, in both Enterprise Managed User and non-Enterprise Managed User scenarios, is what's important, *not* the user's *membership status* in the organization where the scheduled workflow is located.

status

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>status</code>	Not applicable	Last commit on default branch	Default branch

Note

This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when the status of a Git commit changes. For example, commits can be marked as `error`, `failure`, `pending`, or `success`. If you want to provide more details about the status change, you may want to use the `check_run` event. For information about the commit status APIs, see [Commits](#) in the GraphQL API documentation or [REST API endpoints for commits](#).

For example, you can run a workflow when the `status` event occurs.

```
on:
  status
```

If you want to run a job in your workflow based on the new commit state, you can use the `github.event.state` context. For example, the following workflow triggers when a commit status changes, but the `if_error_or_failure` job only runs if the new commit state is `error` or `failure`.

```
on:
  status
jobs:
  if_error_or_failure:
```

```
runs-on: ubuntu-latest
if: >-
  github.event.state == 'error' ||
  github.event.state == 'failure'
steps:
  - env:
      DESCRIPTION: ${{ github.event.description }}
    run: |
      echo The status is error or failed: $DESCRIPTION
```

watch

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>watch</code>	- <code>started</code>	Last commit on default branch	Default branch

Note

- More than one activity type triggers this event. Although only the `started` activity type is supported, specifying the activity type will keep your workflow specific if more activity types are added in the future. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.

Runs your workflow when the workflow's repository is starred. For information about the pull request APIs, see [Activity](#) in the GraphQL API documentation or [REST API endpoints for starring](#).

For example, you can run a workflow when someone stars a repository, which is the `started` activity type for a watch event.

```
on:
  watch:
    types: [started]
```

workflow_call

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
Same as the caller workflow	Not applicable	Same as the caller workflow	Same as the caller workflow

`workflow_call` is used to indicate that a workflow can be called by another workflow. When a workflow is triggered with the `workflow_call` event, the event payload in the called workflow is the same event payload from the calling workflow. For more information, see [Reuse workflows](#).

The example below only runs the workflow when it's called from another workflow:

```
on: workflow_call
```

workflow_dispatch

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
workflow_dispatch	Not applicable	Last commit on the <code>GITHUB_REF</code> branch or tag	Branch or tag that received dispatch

 **Note**

This event will only trigger a workflow run if the workflow file exists on the default branch.

To enable a workflow to be triggered manually, you need to configure the `workflow_dispatch` event. On the GitHub UI, the "Run workflow" button will be present if the workflow file exists on the default branch. Once a workflow has run at least once, you can dispatch it against any branch or tag via the GitHub API or GitHub CLI. For more information, see [Manually running a workflow](#).

```
on: workflow_dispatch
```

Providing inputs

You can configure custom-defined input properties, default input values, and required inputs for the event directly in your workflow. When you trigger the event, you can provide the `ref` and any `inputs`. When the workflow runs, you can access the input values in the `inputs` context. For more information, see [Contexts reference](#).

Note

- The workflow will also receive the inputs in the `github.event.inputs` context. The information in the `inputs` context and `github.event.inputs` context is identical except that the `inputs` context preserves Boolean values as Booleans instead of converting them to strings. The `choice` type resolves to a string and is a single selectable option.
- The maximum number of top-level properties for `inputs` is 25.
- The maximum payload for `inputs` is 65,535 characters.

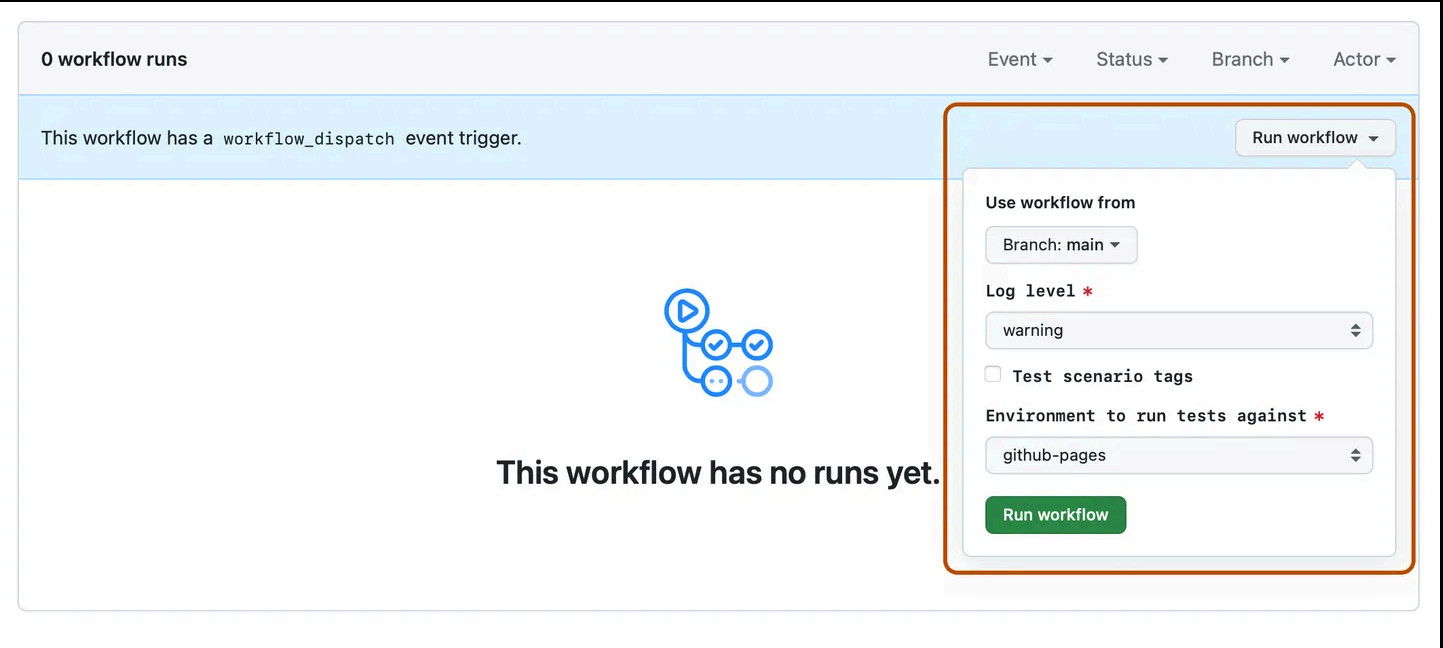
This example defines inputs called `logLevel`, `tags`, and `environment`. You pass values for these inputs to the workflow when you run it. This workflow then prints the values to the log, using the `inputs.logLevel`, `inputs.tags`, and `inputs.environment` context properties.

```
on:
  workflow_dispatch:
    inputs:
      logLevel:
        description: 'Log level'
        required: true
        default: 'warning'
        type: choice
        options:
          - info
          - warning
          - debug
      tags:
        description: 'Test scenario tags'
        required: false
        type: boolean
      environment:
        description: 'Environment to run tests against'
        type: environment
        required: true

jobs:
  log-the-inputs:
    runs-on: ubuntu-latest
    steps:
```

```
- run: |
  echo "Log level: $LEVEL"
  echo "Tags: $TAGS"
  echo "Environment: $ENVIRONMENT"
env:
  LEVEL: ${{ inputs.logLevel }}
  TAGS: ${{ inputs.tags }}
  ENVIRONMENT: ${{ inputs.environment }}
```

If you run this workflow from a browser you must enter values for the required inputs manually before the workflow will run.



You can also pass inputs when you run a workflow from a script, or by using GitHub CLI. For example:

```
gh workflow run run-tests.yml -f logLevel=warning -f tags=false -f environment=staging
```

For more information, see the GitHub CLI information in [Manually running a workflow](#).

workflow_run

Webhook event payload	Activity types	GITHUB_SHA	GITHUB_REF
<code>workflow_run</code>	<code>completed</code> <code>requested</code> <code>in_progress</code>	Last commit on default branch	Default branch

Note

- More than one activity type triggers this event. The `requested` activity type does not occur when a workflow is re-run. For information about each activity type, see [Webhook events and payloads](#). By default, all activity types trigger workflows that run on this event. You can limit your workflow runs to specific activity types using the `types` keyword. For more information, see [Workflow syntax for GitHub Actions](#).
- This event will only trigger a workflow run if the workflow file exists on the default branch.
- You can't use `workflow_run` to chain together more than three levels of workflows. For example, if you attempt to trigger five workflows (named `B` to `F`) to run sequentially after an initial workflow `A` has run (that is: `A` → `B` → `C` → `D` → `E` → `F`), workflows `E` and `F` will not be run.

This event occurs when a workflow run is requested or completed. It allows you to execute a workflow based on execution or completion of another workflow. The workflow started by the `workflow_run` event is able to access secrets and write tokens, even if the previous workflow was not. This is useful in cases where the previous workflow is intentionally not privileged, but you need to take a privileged action in a later workflow.

Warning

Running untrusted code on the `workflow_run` trigger may lead to security vulnerabilities. These vulnerabilities include cache poisoning and granting unintended access to write privileges or secrets. For more information, see [Secure use reference](#) in the GitHub Enterprise Cloud documentation, and [Preventing pwn requests](#) on the GitHub Security Lab website.

In this example, a workflow is configured to run after the separate "Run Tests" workflow completes.

```
on:
  workflow_run:
    workflows: [Run Tests]
```

```
types:
  - completed
```

If you specify multiple `workflows` for the `workflow_run` event, only one of the workflows needs to run. For example, a workflow with the following trigger will run whenever the "Staging" workflow or the "Lab" workflow completes.

```
on:
  workflow_run:
    workflows: [Staging, Lab]
    types:
      - completed
```

Running a workflow based on the conclusion of another workflow [↗](#)

A workflow run is triggered regardless of the conclusion of the previous workflow. If you want to run a job or step based on the result of the triggering workflow, you can use a conditional with the `github.event.workflow_run.conclusion` property. For example, this workflow will run whenever a workflow named "Build" completes, but the `on-success` job will only run if the "Build" workflow succeeded, and the `on-failure` job will only run if the "Build" workflow failed:

```
on:
  workflow_run:
    workflows: [Build]
    types: [completed]

jobs:
  on-success:
    runs-on: ubuntu-latest
    if: ${[ github.event.workflow_run.conclusion == 'success' ]}
    steps:
      - run: echo 'The triggering workflow passed'
  on-failure:
    runs-on: ubuntu-latest
    if: ${[ github.event.workflow_run.conclusion == 'failure' ]}
    steps:
      - run: echo 'The triggering workflow failed'
```

Limiting your workflow to run based on branches [↗](#)

You can use the `branches` or `branches-ignore` filter to specify what branches the triggering workflow must run on in order to trigger your workflow. For more information, see [Workflow syntax for GitHub Actions](#). For example, a workflow with the following trigger will only run when the workflow named `Build` runs on a branch named `canary`.

```
on:
  workflow_run:
    workflows: [Build]
    types: [requested]
    branches: [canary]
```

Using data from the triggering workflow [↗](#)

You can access the `workflow_run` event payload that corresponds to the workflow that triggered your workflow. For example, if your triggering workflow generates artifacts, a workflow triggered with the `workflow_run` event can access these artifacts.

The following workflow uploads data as an artifact. (In this simplified example, the data is the pull request number.)

```
name: Upload data

on:
  pull_request:

jobs:
  upload:
    runs-on: ubuntu-latest

    steps:
      - name: Save PR number
        env:
          PR_NUMBER: ${[ github.event.number ]}
        run: |
          mkdir -p ./pr
          echo $PR_NUMBER > ./pr/pr_number
      - uses: actions/upload-artifact@v4
        with:
          name: pr_number
          path: pr/
```

When a run of the above workflow completes, it triggers a run of the following workflow. The following workflow uses the `github.event.workflow_run` context and the `actions/download-artifact@v5` action to download the artifact that was uploaded by the above workflow, then comments on the pull request whose number was uploaded as an artifact.

```
name: Use the data

on:
  workflow_run:
    workflows: [Upload data]
    types:
      - completed

jobs:
  download:
    runs-on: ubuntu-latest
    permissions:
      actions: read
      issues: write
    steps:
      - name: 'Download artifact'
        uses: actions/download-artifact@v5
        with:
          name: pr_number
          # do not extract in the workspace dir that may contain executable scripts
          path: ${[ runner.temp ]}/artifacts
          run-id: ${[ github.event.workflow_run.id ]}
          github-token: ${[ secrets.GITHUB_TOKEN ]}

      - name: 'Comment on PR'
        uses: actions/github-script@v8
        with:
          github-token: ${[ secrets.GITHUB_TOKEN ]}
          script: |
            const fs = require('fs');
            const path = require('path');
            const temp = `${[ runner.temp ]}/artifacts`;
            const issue_number_raw = fs.readFileSync(path.join(temp, 'pr_number'),
            'utf8').trim();
            const issue_number = Number(issue_number_raw);
            if (!Number.isInteger(issue_number)) {
              throw new Error(`Invalid PR number in pr_number artifact:
            "${[ issue_number_raw ]}"`);
            }
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
```

```
issue_number: issue_number,  
body: 'Thank you for the PR!'  
});
```



Back to top



Was this Doc helpful?

Yes

No

Help us make GitHub Docs great!

All Docs are open source. See something that's wrong or unclear? Submit a pull request.

Make a contribution

STILL NEED HELP?

- Ask the GitHub community
- Contact support
- Expert services
- Blog